Working with large models

10-701 Introduction to Machine Learning
Geoff Gordon

with thanks to Matt Gormley

Reuse

- Attention on long contexts is expensive!
  - big matrix multiplies: saturates memory bandwidth
- But the exact same computations show up multiple times in real workloads → can save and reuse results
  - Example 1: many users ask the exact same question (same prompt) — or a refinement of the same question (one prompt is a prefix of another)
  - Example 2: through the course of a conversation, each prompt has previous prompt as a prefix
  - Example 3: we have 1000 token prompt and 300 token response; response token 301 needs all the same matrix computations as token 300 did (plus some more)

Caching

- Treat as a caching problem: save results in multiple levels of storage
  - each level larger but slower than previous one
  - when a level gets full, evict (bump down to slower storage) the results that we are least likely to reuse
  - or drop entirely once largest/slowest level fills up
- Unit of reuse: (key, value) pairs at each attention layer
  - expensive: needed forward pass up to that layer
  - worth saving: future tokens attend over past keys/values
  - note: past queries aren't reused (generating a new token needs *this token's* queries and *past tokens'* keys/values)
- Unlike some caches, can't evict independently: attention needs all previous tokens to generate next

KV cache and prefix cache

- Two important cases:
  - ***KV cache**:* previous tokens in the current generation
  - ***prefix cache**:* previous generations
- Often share storage in a single caching system, but different properties (e.g., mutability), so different treatment

Prefill vs generation

- Two distinct workloads:
  - ***prefill****:* process a long context
  - ***generation****:* produce new tokens one at a time
- Prefill is much easier to parallelize: tokens are all known ahead of time, no need to complete computation for position i before starting to process position $i+1$
  - e.g., add a new document to our database for RAG
  - or process a document for classification (e.g., sentiment of a review)
- Generation seems inherently sequential, but …

Speculative decoding

the mayor of New

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Speculative decoding

the mayor of New York City, Zohran Mamdani, appeared at an event

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Speculative decoding

York City, Zohran Mamdani, signed at an event

the mayor of New York City, Zohran Mamdani, appeared at an event

- To improve latency of generation (at some cost to total work), we'd like to generate multiple tokens at once
- Idea: generate a few tokens (say, 5–10) from a cheaper model — this can be extremely fast
  - cheaper model will often be right, especially in easy cases (like filling in boilerplate arguments to a function)
- Then verify these tokens in parallel w/ expensive model
  - i.e., generate each position conditioned on previous positions being correct
  - keep an initial segment of tokens that agree — once the simpler model makes a mistake, have to discard all future tokens
- We win as long as simpler model is right often enough

Language model as a classifier

- LLMs provide a highly informative prior for supervised learning tasks: trillions of tokens of human knowledge
- Suppose we can afford just 10 labeled examples in a new classification task; can we harness the LLM's prior to generalize well?

Representing examples for the LLM

- To take advantage of LLM's knowledge, need to transform (x, y) pairs into a format it can understand
- For inputs: write a short bit of code that turns features into natural language
  - e.g., `(red, 12)` → "the product is red and 12 cm tall"
- Optionally add a descriptive prompt:
  - "will it be a good gift for a kid?"
- For outputs:
  - pick appropriate tokens to represent the classes: e.g., "yes" and "no" for a binary classifier
  - to read out the answer, prohibit all other tokens: just test whether $P(\text{yes} \mid \text{input}) > P(\text{no} \mid \text{input})$

Fine-tuning

- First approach: fine-tuning
- Just like the fine-tuning examples above, continue training the pre-trained model on our new (tiny) dataset
  - use supervised loss (e.g., cross-entropy)
  - seems crazy that it can work with so few examples!
  - typically only need small learning rate and few epochs
- This variant is ***pattern-based*** fine-tuning
  - for small datasets, works better than other FT variants such as adding a new prediction head to the network

In-context learning

- Second approach: in-context learning
- Just include training examples in prompt before asking about test example:

```
red, 12 cm tall? Yes
blue, 15 cm tall? No
. . .
orange, 9 cm tall?
```

training examples

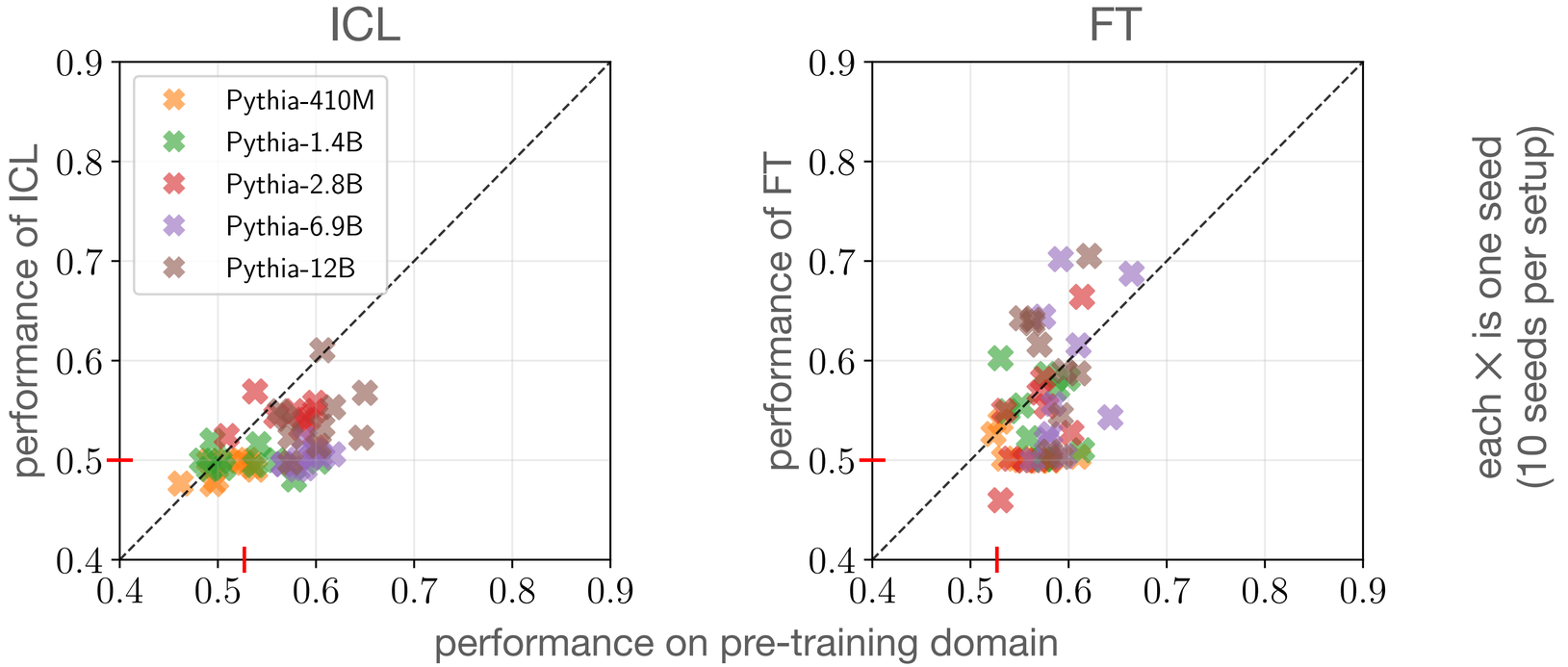
test input

- As above, read answer as $P(\text{yes} \mid \text{input}) > P(\text{no} \mid \text{input})$
- Huge advantage: no training needed!
  - fast, convenient
  - no need to know weights (e.g., API-only access)

[Mosbach et al., 2023]

Comparison

Graphs: 16 training examples, textual entailment task, models of multiple sizes with the same data (the Pile) and training pipeline



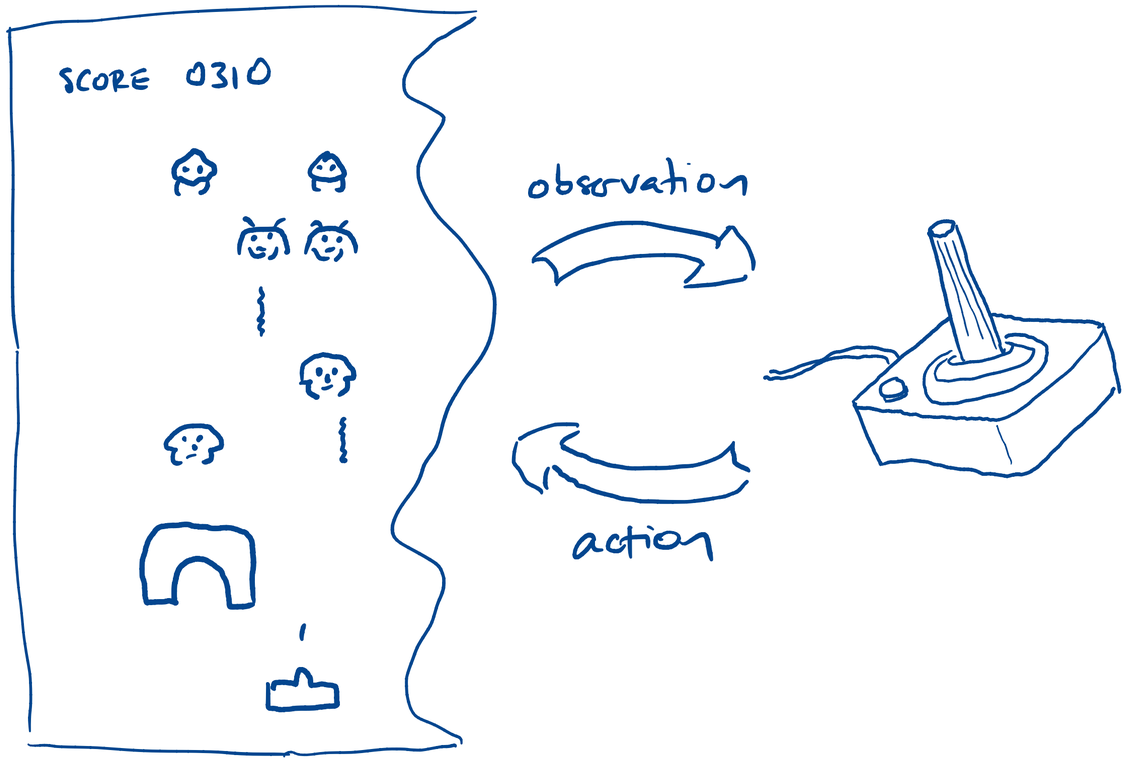
- ICL is fast and convenient, and can perform well for small training set sizes
- FT tends to win as we get more training data
- Both do better when base model is better — crossover point depends on base model
  - the better the model, the longer ICL remains competitive

Reinforcement learning

10-701 Introduction to Machine Learning

Geoff Gordon

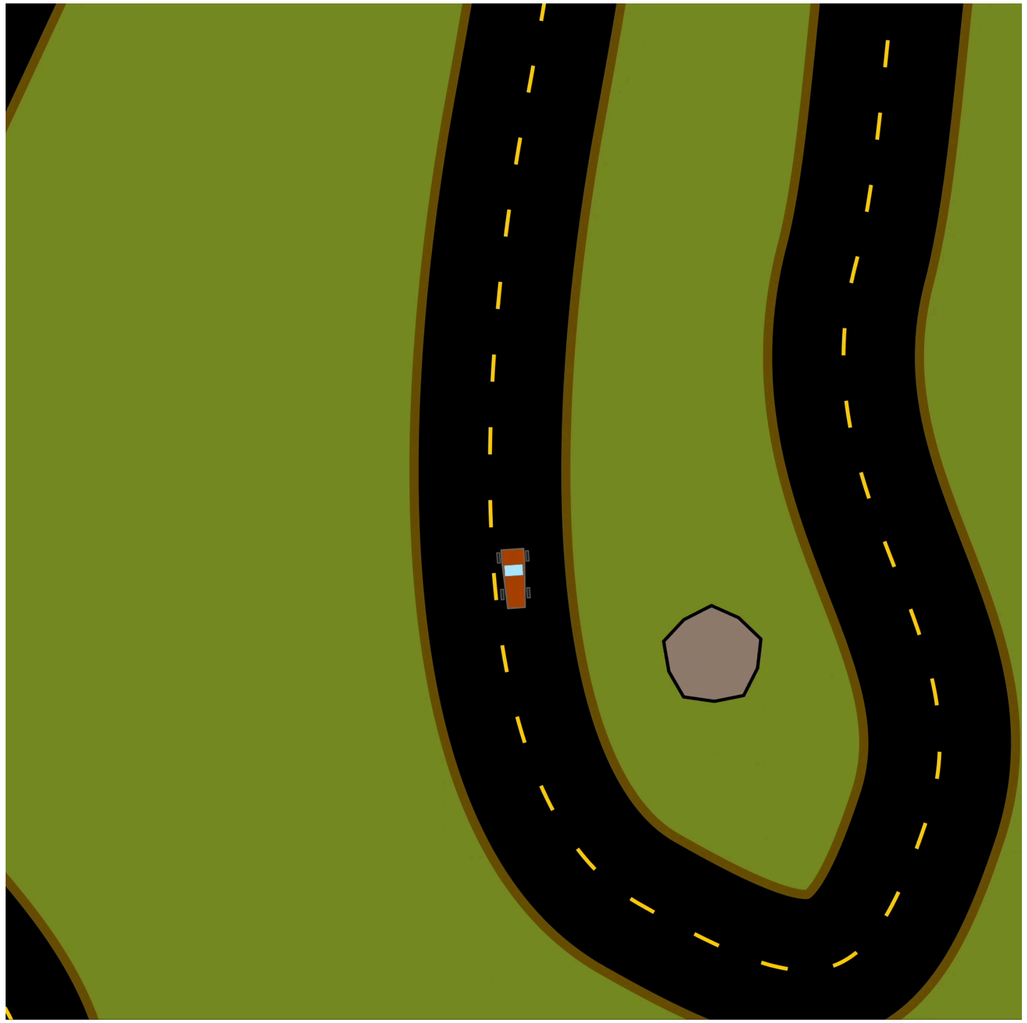
Sequential decision problem



- Agent interacts w/ environment over time
- Alternating observations and actions $o_1, a_1, o_2, a_2, \ldots$

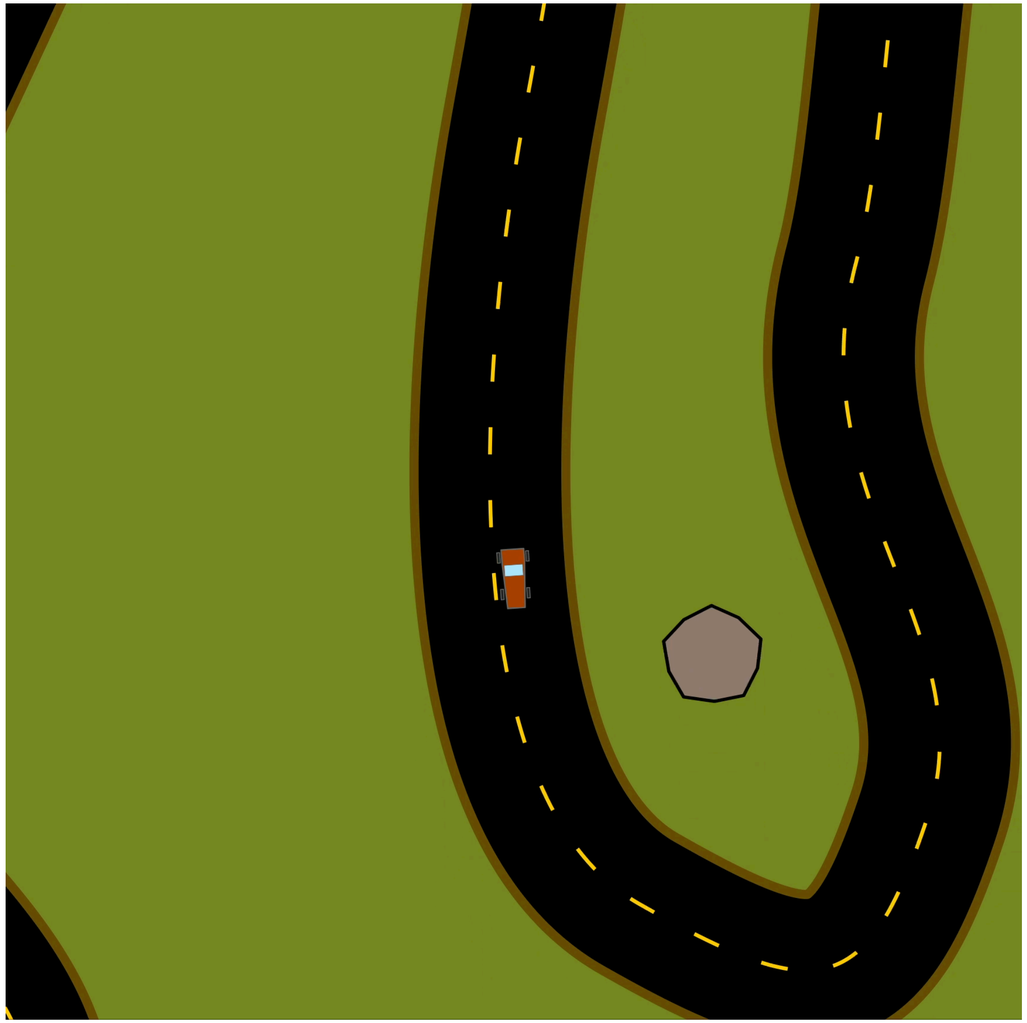
called a *trajectory*

Example: simple racing game



- observations: 60 fps images, downscale to 32×32
- actions: 2D real = offset from car center to steering target
- trajectory: images and steering over time

Example: simple racing game

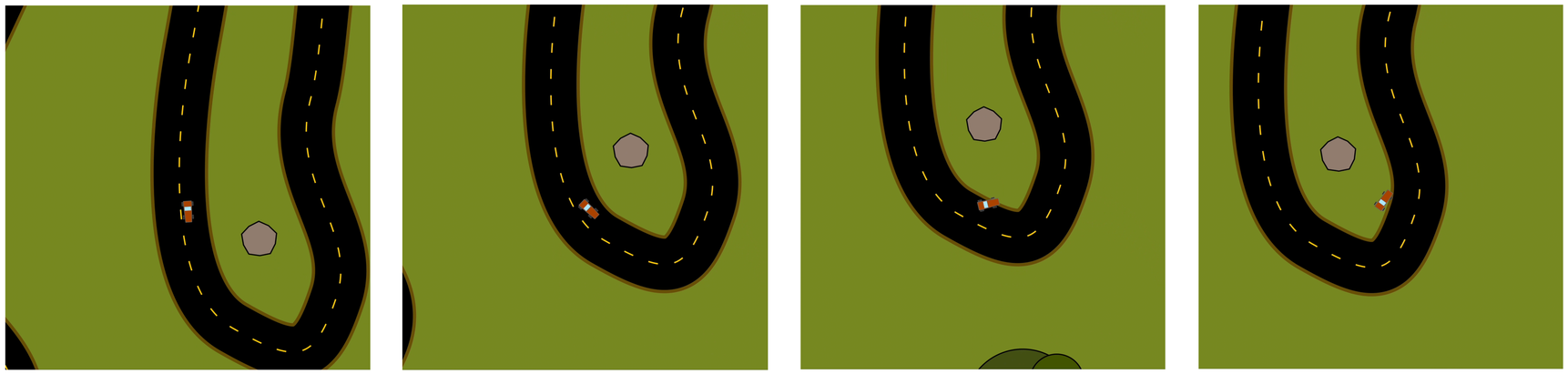


- observations: 60 fps images, downscale to 32×32
- actions: 2D real = offset from car center to steering target
- trajectory: images and steering over time

Reinforcement learning

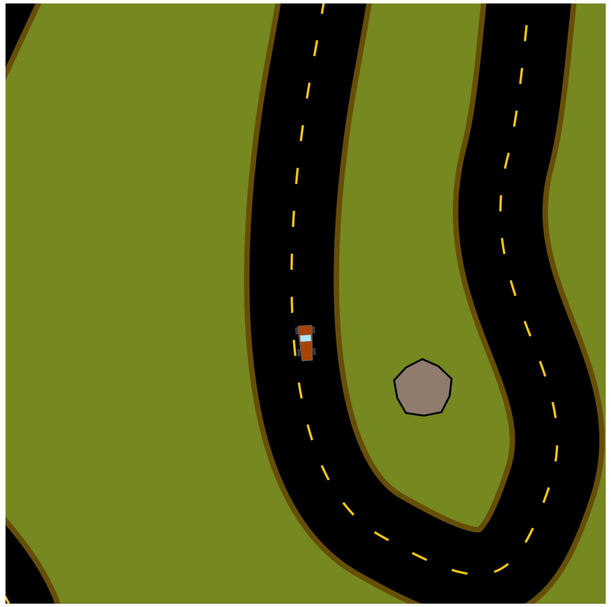
- Many possible goals in a sequential decision problem: environment modeling, reinforcement learning, imitation, apprenticeship, behavior design
- Warning: we'll cover one goal here (RL); popular but often not the right way to approach a problem
- Assume: given *one-step* costs or rewards: $c_t = c(o_t, a_t)$
- RL: act to maximize total reward or minimize total cost
  - say, sum to end of level (cross finish or pass time limit)

$$\min_{\text{agent code A}} \mathbb{E}_{\text{trajectory } \tau \mid \text{A}} \text{ total cost of } \tau$$



cost=0 cost=0.1 cost=0.5 cost=1.1

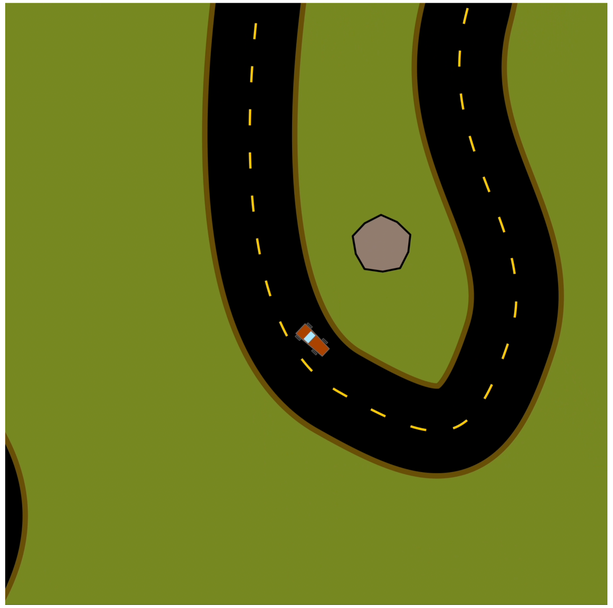
costs determined by *current* observation and action — if not, update observation features



Stay on track: cost=0

Go straight: cost=0

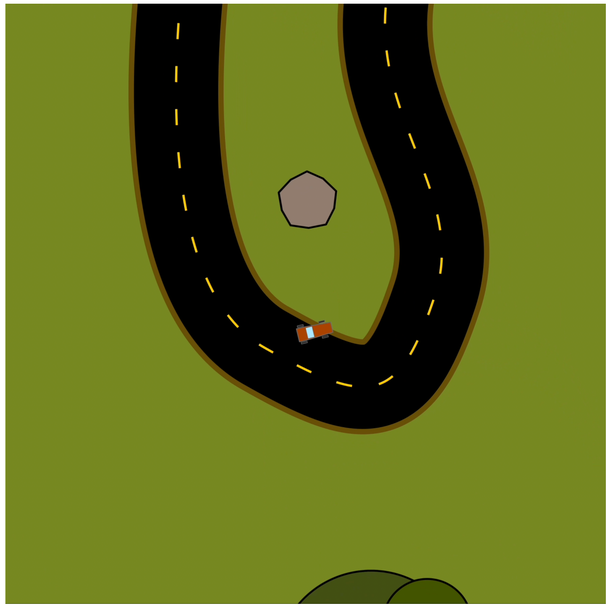
Go fast: cost=0



Stay on track: cost=0

Go straight: cost=0.7

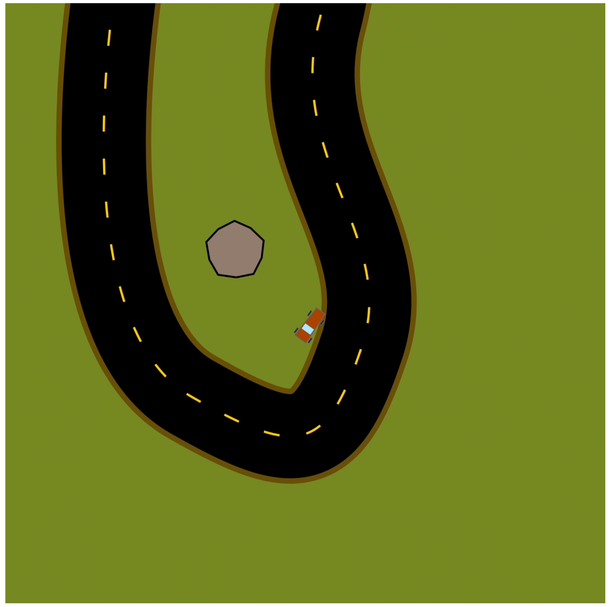
Go fast: cost=0.3



Stay on track: cost=0.5

Go straight: cost=1

Go fast: cost=0.7



Stay on track: cost=1

Go straight: cost=0.7

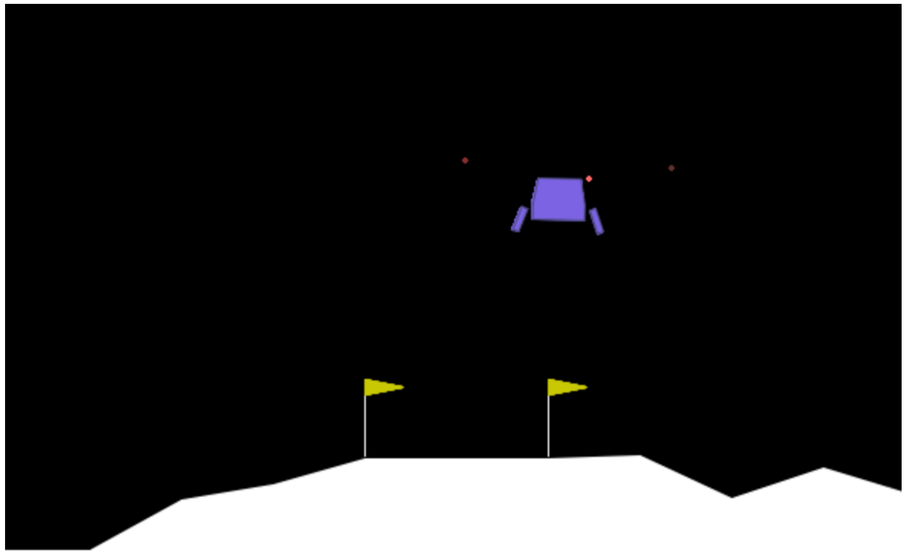
Go fast: cost=1

One-step cost

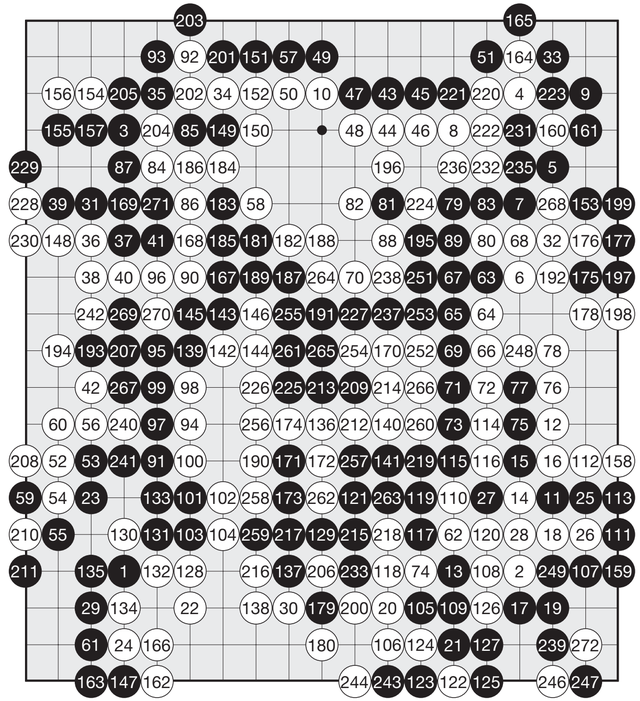
- Many possible cost/reward functions: stay on track, go fast, avoid sharp turns, …
- Choosing the right one is a hard problem in itself!
  - common source of bugs: reasonable-seeming cost/reward leads to unreasonable behavior

Example environments

Sometimes we consider the goal (cost/reward) as part of the environment, but sometimes the same environment could support more than one RL problem



observations: screen images
actions: controller buttons, joystick position
transitions: determined by game code
reward: score increase



observations: board $\{B, W, \varnothing\}^{19\times19}$
actions: place a stone
transitions: rules of Go, opponent follows a previous policy (self-play)
reward: +1 for win, –1 for loss, 0 for draw, 0 if game isn't over

Why is RL hard?

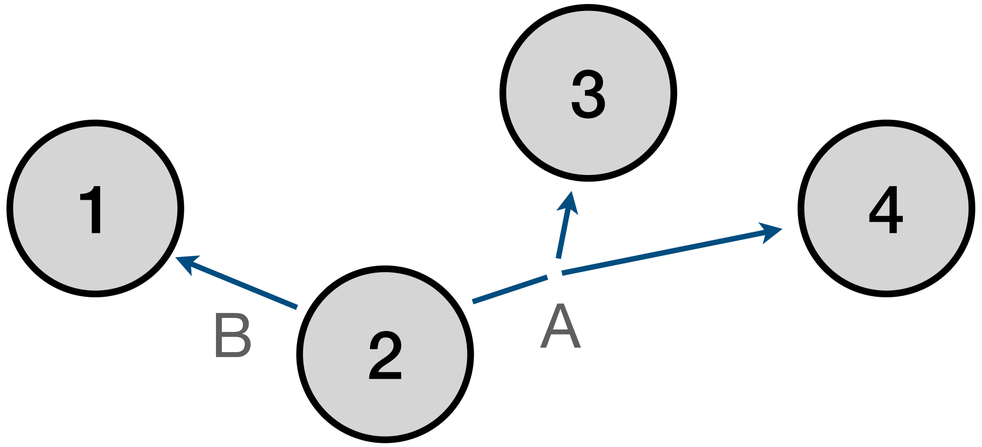
- Some key difficulties make RL a lot harder than supervised learning
  - the ***explore-exploit dilemma***
    - do we try to learn more about the world so that we can do better later, or do we try to get low cost now?
  - the ***temporal credit assignment problem***
    - if we lose the game, where was our mistake?
  - the ***exposure bias problem***
    - if our learner makes a mistake in a rare situation, it could turn out to be important — future choices could make the situation much more common

Warmup: explore-exploit

- If we fix the horizon (max episode length) to 1, RL becomes the ***contextual bandits*** problem
  - agent sees context o (an observation, independent from all previous contexts and actions)
  - chooses action a
  - gets reward r
  - repeat
- E.g.:
  - recommend a news article
  - pick ads to serve on a search page

Warmup 2: tabular setting

Both explore-exploit and temporal credit assignment, but no worry about how to approximate high-d functions



- Tractable set of observations {1, 2, 3, 4} × actions {A, B}
- The ***Markov property***
  - we only need to remember the most recent observation
  - $P(o_{t+1} \mid o_1, a_1, \ldots, o_t, a_t) = P(o_{t+1} \mid o_t, a_t)$
  - in this case, o_t is called a ***state***, often write s_t instead
- Why “tabular”? OK to store everything as tables: e.g., table of costs or table of best actions to take

Warmup 2: tabular setting

Both explore-exploit and temporal credit assignment, but no worry about how to approximate high-d functions

Don't know full environment to start:
only $\mathcal{S} = \{1 \ldots 4\}$ and $\mathcal{A} = \{A, B\}$

- Tractable set of observations {1, 2, 3, 4} × actions {A, B}
- The ***Markov property***
  - we only need to remember the most recent observation
  - $P(o_{t+1} \mid o_1, a_1, \ldots, o_t, a_t) = P(o_{t+1} \mid o_t, a_t)$
  - in this case, o_t is called a ***state***, often write s_t instead
- Why "tabular"? OK to store everything as tables: e.g., table of costs or table of best actions to take

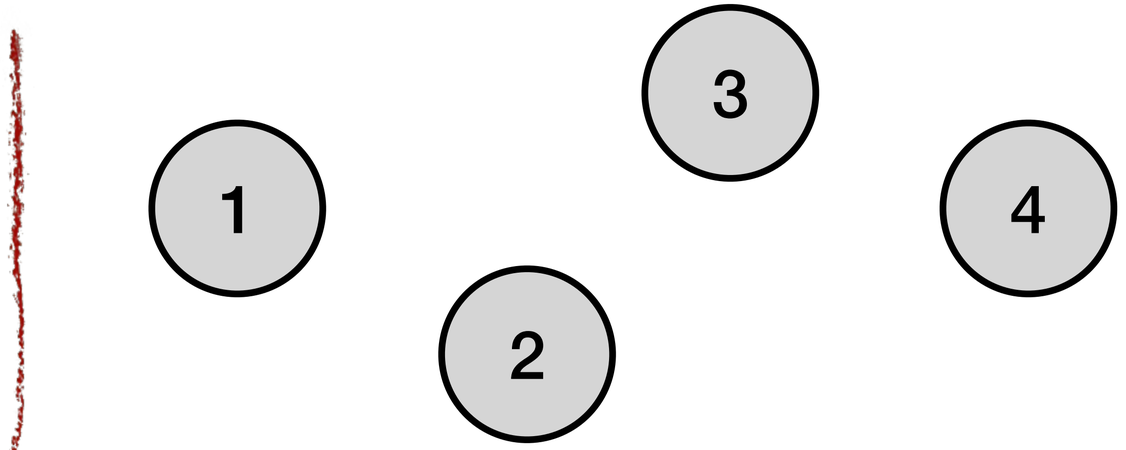
Collect data, learn $P(s_1)$***,*** $r(s, a)$***, and*** $T(s' \mid s, a)$

$P(s_1)$: initial state dist'n

$r(s, a)$: one-step reward

$T(s' \mid s, a)$: transition prob

note: ***substochastic***



start state

1	2	3	4

4 states {1, 2, 3, 4}
2 actions {A, B}

reward count

to state

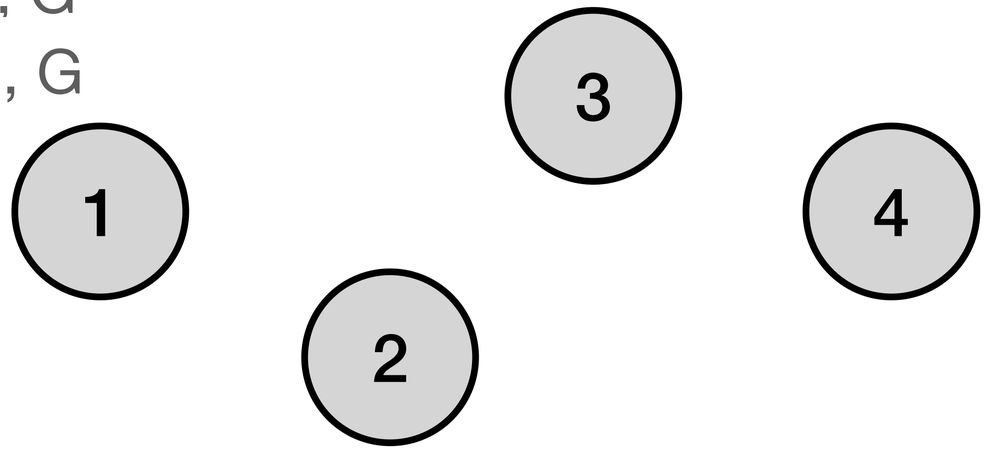
from state/act	1	2	3	4
1 A				
1 B				
2 A				
2 B				
3 A				
3 B				
4 A				
4 B				

estimated $P(s' \mid 4, A)$:

τ_1: 4, A, 4, A, 1, A, 2, A, 3, A, G

τ_2: 3, B, 2, B, 1, B, 4, B, 3, B, G

$r = 0$ each step
except last $r = 1$



start state

1	2	3	4

4 states {1, 2, 3, 4}
2 actions {A, B}

Collect data, learn $P(s_1)$***,*** $r(s, a)$***, and*** $T(s' \mid s, a)$

reward count

to state

from state/act

		1	2	3	4
1	A				
	B				
2	A				
	B				
3	A				
	B				
4	A				
	B				

estimated $P(s' \mid 4, A)$:

$P(s_1)$: initial state dist'n

$r(s, a)$: one-step reward

$T(s' \mid s, a)$: transition prob

note: ***substochastic***

Names for environment types

- If max trajectory length is 1: contextual bandit
- If no choice of action: hidden Markov model (HMM)
- If observation is a state: Markov decision process (MDP)
  - called ***observable*** or ***fully-observable***
- If not: partially-observable MDP (POMDP)
- If observation and action are continuous: control problem

Time as state

- In our tabular examples so far, environment dynamics don't change with time
  - the environments are ***stationary***
- If we want a nonstationary environment (e.g., a deadline by which something has to happen, or a bonus for doing something early), just include the time index as part of the state
  - $T(s' \mid s, a)$ will increment it
  - finite $\mathcal{S}$ means there's a hard time limit — the horizon
  - we'll effectively keep one set of tables per time step
- Or, can have an approximate time index: e.g., a counter that increments w.p. $\frac{1}{2}$ and maxes at 20 means we know approximate time for ~40 steps, then "it's been a while"

Agent behavior = policy

- Policy = function π
  - input: history (trajectory so far) $o_1, a_1, \ldots, o_t$
  - output: $P(\text{action} \mid \text{history})$
- With Markov assumption:
  - enough to depend just on state (last observation)
  - write $\pi(a_t \mid s_t)$
  - optimal policy can be deterministic, but we typically optimize over stochastic policies (continuous set)
- Optimal answer: policy with best expected (total) reward

Policy

State	***P(left)***	***P(right)***
1		
2		
3		
4		
5		

Estimating the total reward of a policy

- Naive method: run some trajectories with agent acting according to policy π, average their total rewards
- Insight: the actual sequence of states visited is informative — use it to reduce variance
- We'll estimate rewards for each state, use relationships among states to improve accuracy
- ***Defn**:* the ***value function*** of π maps state $\rightarrow$ total expected future reward if we are at that state following π

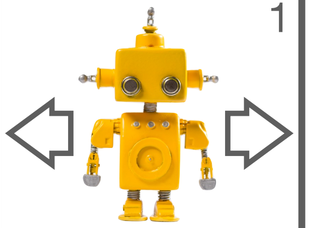
$$V^\pi(s) = \mathbb{E}\left[\sum_{i=t,\,t+1,\,\ldots} r_i \,\middle|\, s_t = s, \text{following } \pi\right]$$

  - for now, assume expectation exists; we'll return to this

Value function example

For simplicity, suppose we know the exact environment

Environment

1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

Policy π: always move right

Trajectory ends with any action from 5

Table of V^π

1	2	3	4	5
–13	–12	–11	–1	0

Computing the value function

$$
\begin{aligned}
V^\pi(s) &= \mathbb{E}(r_1 + r_2 + r_3 + \ldots \mid s_1 = s, \text{following } \pi) \\
&= \mathbb{E}(r_1 + \mathbb{E}(r_2 + r_3 + \ldots \mid s_2 = \text{observed } s_2, \pi) \mid s_1 = s, \pi) \\
&= \mathbb{E}(r_1 + V^\pi(s_2) \mid s_1 = s, \pi) \\
&= \sum_a \pi(a \mid s) \left[r(s,a) + \sum_{s'} T(s' \mid s, a) V^\pi(s') \right]
\end{aligned}
$$

note: T is substochastic — handles end of trajectory automatically

- Split $V^\pi(s)$ into first step and subsequent steps
- Use law of iterated expectations + Markov property
- Result: V^π is expressed recursively (in terms of itself)
  - called the ***Bellman equation***
  - will allow a ***dynamic programming*** algorithm

Value iteration for V^π

Dynamic programming:
replace = by ← in Bellman equation

Environment

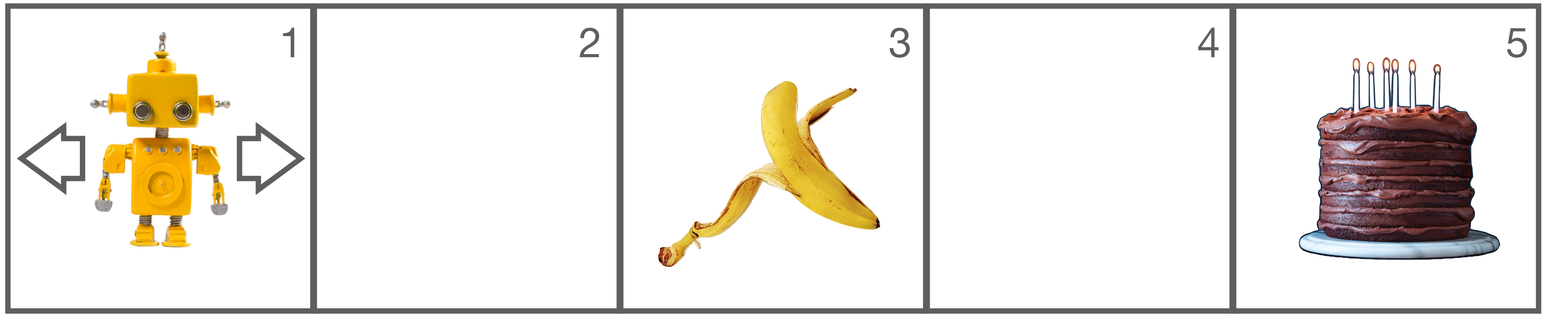


Table of V^π

1	2	3	4	5

- Given: $r(s,a)$, $T(s' \mid s,a)$ (exact or learned)
- Initialize $V^\pi(s)$ arbitrarily (e.g., to 0 for all s) — or warm start if we know one
- Repeat until converged
  - for each state s (in parallel or in an arbitrary order)

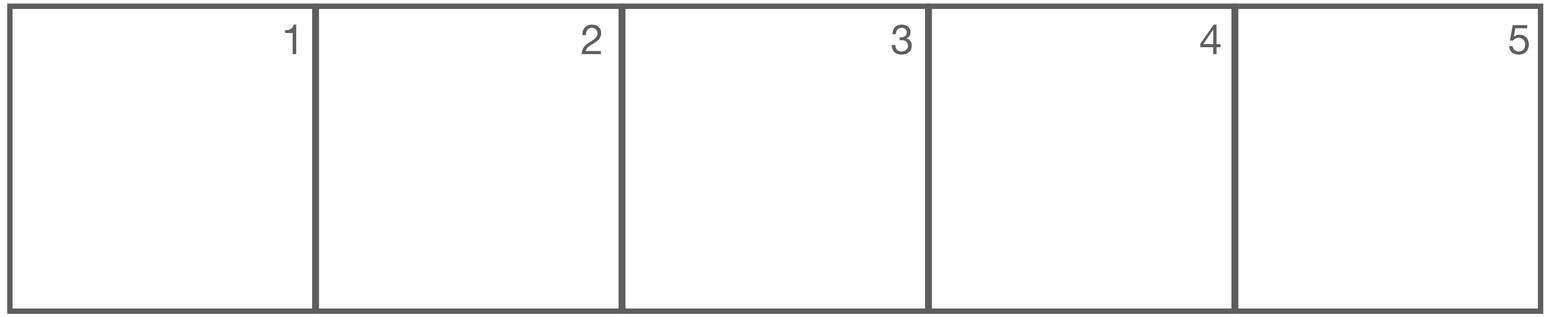
$$V^\pi(s) \leftarrow \sum_a \pi(a \mid s)\left[r(s,a) + \sum_{s'} T(s' \mid s,a) V^\pi(s')\right]$$

Temporal difference (TD) learning

same idea, but from data instead of from environment model

Data $1 \to^{-1} 2 \to^{-1} 3 \to^{-10} 4 \to^{-1} 5 \to^{0}$ goal (π: always R)

Table of V^π



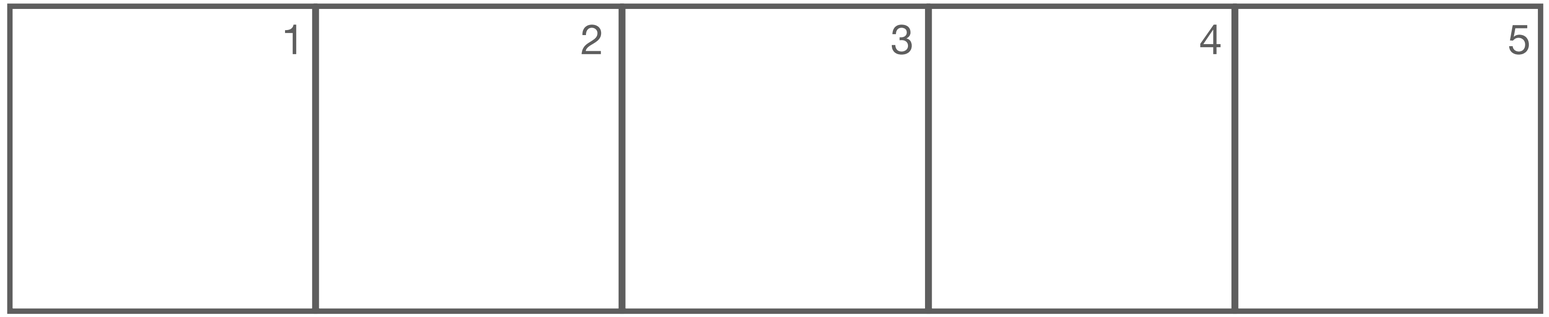
- Given: observed transitions from π, $\{s_t, a_t, r_t, s_{t+1}\}$
- Initialize $V^\pi(s)$ arbitrarily (e.g., to 0 for all s)
- Save a frozen copy $V^\pi_{\text{tgt}}(s)$
  - i.e., we stop gradients for V^π_{tgt} and don't update in SGD
  - below, we'll update V^π_{tgt} every so often

Temporal difference (TD) learning

same idea, but from data instead of from environment model

Data $1 \to^{-1} 2 \to^{-1} 3 \to^{-10} 4 \to^{-1} 5 \to^{0}$ goal (π: always R)

Table of V^π



- [main loop] Repeat for a while
  - sample a transition s, a, r, s' (or minibatch)
  - compute target values $r + V^\pi_{\text{tgt}}(s')$
  - update each $V^\pi(s)$ by SGD towards target values:

```
loss += 0.5 * (V[s] - target)**2
loss.backward(); optim.step()
```

- Update targets $V^\pi_{\text{tgt}} \leftarrow V^\pi$ and go back to [main loop]

Optimal value function

Def'n:

$$V^*(s) = \max_\pi \mathbb{E}(r_1 + r_2 + r_3 + \ldots \mid s_1 = s, \text{following } \pi)$$

$$V^*(s) = \max_a \mathbb{E}\left[r_1 + \max_\pi \mathbb{E}(r_2 + r_3 + \ldots \mid s_2 = \text{observed } s_2, \pi) \mid s_1 = s, a_1 = a)\right]$$

$$= \max_a \mathbb{E}\left[r_1 + V^*(s_2) \mid s_1 = s, a_1 = a\right]$$

$$= \max_a \left[r(s,a) + \sum_{s'} T(s' \mid s, a) V^*(s')\right]$$

- Decompose $V^*(s)$ into first step and subsequent steps, using law of iterated expectations as above
- Split $\max_\pi$ into separate max over a_1 and over π for $t = 2$ onward
- Result: V^* expressed recursively (another Bellman eq)

Value iteration for V^*

Replace = by ← in Bellman equation

Environment

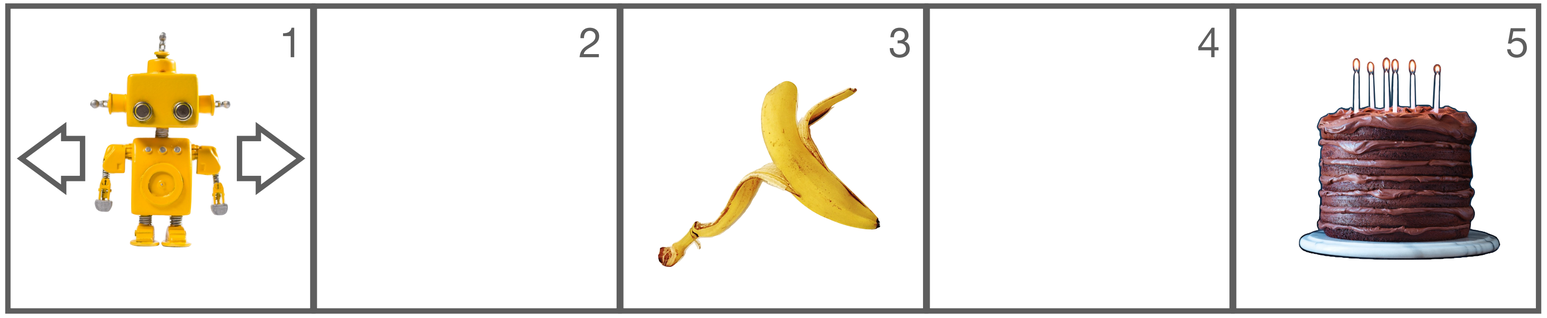


Table of V^π

1	2	3	4	5

- Given: $r(s,a)$, $T(s' \mid s,a)$
- Initialize $V^*(s)$ arbitrarily (e.g., to 0 for all s)
- Repeat until converged
  - for each state s (in parallel or in an arbitrary order)

$$V^*(s) \leftarrow \max_a \left[r(s,a) + \sum_{s'} T(s' \mid s,a) V^*(s') \right]$$